ARTICLE

# From information visualization to direct manipulation: extending a generic visualization framework for the interactive editing of large datasets

**THOMAS BAUDEL**, International Business Machines, Armonk, NY, United States



**Published:** 15 October 2006

**Citation in BibTeX format**

UIST06: The 19th Annual ACM Symposium on User Interface Software and Technology
*October 15 - 18, 2006*
*Montreux, Switzerland*

**Conference Sponsors:**
SIGCHI
SIGGRAPH

# From Information Visualization to Direct Manipulation: Extending a Generic Visualization Framework for the Interactive Editing of Large Datasets

*Thomas Baudel*
ILOG, 9 rue de Verdun
94253 Gentilly Cedex, France
baudel@ilog.fr

**ABSTRACT**

Today's generic data management applications such as accounting, CRM or logging and tracking software, rely on form and menu based interfaces. These applications take only marginal advantage of current graphical user interfaces. This is because the data they handle does not have intrinsic visual representations upon which direct manipulation principles can be used. This article presents how we have extended an Information Visualization framework with generic data manipulation functions. These new data editing capabilities are tuned to take advantage of the characteristics of each view. They enable us to generalize the direct manipulation mechanisms to address many abstract data manipulation needs. In this article we present five uses of the features we have implemented and deduce a general workflow applicable to a variety of contexts. The workflow comprises three steps and five editing actions. The steps are: adjust view, select, and edit. The editing actions are: edit a value or group of values, clone objects, remove objects, add attributes, and remove attributes. The workflow provides complete editing access to table and hierarchical data structures using particularly terse interaction methods. It defines a general data editing model that enables powerful data manipulation tasks without requiring end-user programming or scripting.

**ACM Classification**

H5.2 [Information interfaces and presentation]: User Interfaces - Graphical user interfaces, Interaction styles, I3.6 - Interaction techniques, Graphics data structures and data types.

**Keywords**

Information Visualization, Direct Manipulation, Direct Data editing, database user interfaces.

**INTRODUCTION**

The benefits of direct manipulation [18] are widely acknowledged in many domains of interactive computing. This includes the fields of desktop publishing, computer graphics, or visual data analysis. However, all sorts of office automation applications use only form and menu based interaction augmented with list and table displays for elementary information retrieval. Indeed, a simple text-based terminal is amply sufficient to perform the daily tasks involved in, for instance, accounting or customer relationships management. It is also sufficient for most of the online search and purchase activities performed on the web. Current graphical user interfaces for these tasks provide additional comfort and attractiveness, but only marginal efficiency improvements.


*UIST'06,* October 15–18, 2006, Montreux, Switzerland.


Direct manipulation provides efficient and intuitive interaction methods only when the application domain allows the creation of interactive and graphical representation of domain objects. Common data objects such as invoices, customer profiles, or logging data do not have intrinsic visual representations, and therefore have no "natural" direct manipulation editing methods.

Yet, traditional data management applications do need improved user interfaces to provide easier access to larger information volumes. These enhanced interfaces need to provide in-context, fast, incremental, and reversible actions on the subsets of interest. This is because firstly, the volume of available data to take into account and manage grows by orders of magnitude and secondly, because interconnected systems and online services reduce routine data entry tasks and replace them with analysis and decision making activities. These new activities require that we take more of the context into account and study alternative hypotheses in data-rich universes.

To address these requirements, we need to provide direct manipulation methods adapted to handle abstract data. Information Visualization, as a field, provides the first answer to this issue. It does so by providing the means to create rich and intuitive data representations. However, interactions provided by existing Information Visualization packages are limited to navigation features, including geometric (zooming, scrolling...), topological (focus-in or out, follow links and paths...), or algebraic (select, join, project...) navigation. By definition, visual data analysis systems do not focus on how to edit the data at hand.

This article shows how to augment Information Visualization solutions with generic information manipulation mechanisms that rely on the visualization of complete data

sets. These mechanisms comply with the first 2 principles of direct manipulation, namely:

- Provide persistent representations of the objects of interest and the available actions.
- Provide fast, incremental, and reversible actions on the objects of interest or some selected subsets.

Because representations created by a visualization system are necessarily abstract, they cannot rely on a metaphor or on a convention known to the user. However, the tools we propose enable the user to modify the data representation at will, and to present a data set under the most appropriate angle for a given task.

The editing actions available on a data set can be deduced from the chosen representation, and from conventional graphic manipulations such as select, move, and resize found in direct manipulation applications such as drawing or text editors. This provides a simple, generic action vocabulary that can be used to perform a wide variety of operations whose outcome remains predictable to the user.

To implement this editing paradigm, we introduce a generic *data manipulation framework*. This framework is used to describe or infer the actions available on a view. It is also used to see how actions translate into data modification. This generalizes the concept of direct manipulation to data sets that do not have an intrinsic visual representation. The general editing model allows all data editing to be performed by a combination of three steps and five editing actions performed in any desired order. The steps are:

- **Adjust** the view settings to make the representation suit the operations to be performed.
- **Select** the objects or groups of objects of interest by means of regular or extended selection mechanisms.
- **Perform** one of the five editing actions. That is, change a value, clone the selection, delete the selection, add or remove an attribute to/from the selection.

After placing our research in context, we present five examples of direct data manipulation. From these examples we draw general design principles and propose an interaction model for generic data manipulation. *This model uses the properties of the view to deduce the semantics of the user's action.* Finally, we discuss the limitations as well as the perspectives opened by our interaction model.

## VISUALIZATION FRAMEWORKS

This research stems from a design evolution of the generic Information Visualization framework, ILOG Discovery [2]. This framework is not centered on predefined visualization algorithms like GGobi, Spotfire or Xmdv are. Instead it allows the interactive creation and manipulation of visualization algorithms like Visualization Spreadsheets [5] do.

The framework relies on a dataflow model. Like many similar software packages such as VTK [19], VisAd [11], or OpenDX [21]: transformation operators placed in sequence successively transform data objects into intermediate objects. These objects are ultimately turned into visual representations, themselves mapped on a display according to appropriate view transformations. Several parameters control each step of the transformation. This enables the system to provide a wide range of real-time adjustable visualizations. Like similar frameworks such as prefuse [10] or the InfoViz toolkit [9], ILOG Discovery enhances the dataflow with a set of interactive features aimed at aiding visual data analysis tasks.

The originality of ILOG Discovery stems from its use of linear-state-dataflows. These involve only 4 operators, which are applied in the following fixed order:

1. **Partitioning** a data set into groups and subgroups.
2. **Sorting** groups and individual objects.
3. **Assigning graphic primitives** to groups or objects.
4. **Decorating** graphic primitives. That is, assigning color and other graphic attributes.

All dataflow parameters are programming language expressions involving constants, data attributes, local variables, and accumulators. These two last concepts provide a means to augment the dataflow with state-handling capabilities. They are inspired from the co-routine mechanism of the CLU programming language [16]. The groups resulting from a partition can be represented recursively using child dataflows of the same kind. Like Polaris [20] or nVizN [22], this model defines an algebra of visualization algorithms that exhibits useful properties. For example, it can be shown that linear-state-dataflows provide a canonical representation for a large class of visualization algorithms, called "data-linear visualizations", which corresponds roughly to the visualization algorithms that perform at most a fixed number of passes through the data set.

The precise semantics of our model enables the user to describe a very wide variety of views using only a minimal number of parameters. The model lets one describe all sorts of views such as charts, 2 plots, and treemaps with a uniform set of parameters, and enables mixing visualization features to provide an endless variety of visualizations, requiring the user to master only a few general principles. We use this power of expression to enable users to switch rapidly between various representations, and to easily find those representations that enable the selection and editing of the objects they are most interested in. This model, its main properties, and implementation details are described in [1]. It is prerequisite to understand [1] to grasp the implementation details provided thereafter.

## INTERACTIVE DATASET EDITING

As stated earlier, most database editing tasks are performed with menu and form based user interfaces. However, in some cases, direct manipulation functionalities are used to edit rich datasets. For example, many planning applications enable the reworking of plans and schedules by direct manipulation of activities and resources in Gantt or Pert charts. The techniques proposed by those applications are useful, but do not constitute a global editing scheme that can be used in a variety of contexts. Spreadsheet programs

and some database management tools enable users to edit datasets in bulk. However, except for a few predefined functions, bulk operations are performed programmatically with macros rather than through direct manipulation.

The concept of "Programming by example" [6, 15] is closer to the editing model we propose. In these experimental systems, the semantics of the user's actions are derived by interpreting graphical editing activities using heuristic rules. In our case, no heuristic is involved in determining the user's intent. Attributes are editable only if the view settings enable the software to unambiguously determine the objects and attributes impacted; otherwise, no interpretation of the possible outcome of a graphical action is performed. In this respect, our work is also inspired by the domain of graphical constraints inference, such as RockIt [12].

The Sage/Visage [4, 14, 17] research bears many similarities to our work, in particular the Interactive Visual Query Environment (VQE) [7, 8]. This environment describes ways to infer semantic queries based on the manipulation of graphic objects and the view definition. VQE does not rely on an algebra of operators as concise and complete as the one we use: the partitioning operator, recursively applied dataflows, and local variables are missing from VQE. These concepts offer alternatives to designate and transform groups of objects. Hence the proposed mechanisms do not bear the same generality and have not been extended as a general interaction model usable for data editing.

## INTERACTIVE CLEANSING AND WHAT-IF ANALYSIS

The first editing task performed in data analysis is data cleansing. Existing visualization systems are usually able to detect missing data and entry errors quite easily. It is, therefore, quite surprising that none of these systems integrate a means of correcting erroneous data seamlessly through direct manipulation as part of its regular feature set.

In our framework, data attributes can be edited using regular graphic interactions, such as moving or resizing objects in a continuous geometric space. The only necessary condition is that the mapping function between the attributes of a data object and its graphic representation (step 3 of the dataflow described above) must be an invertible formula.

As an introductory example, Figure 1 presents a set of vehicles, plotted in a Weight versus Miles Per Gallon (MPG) graph. Moving selected items in this graph has a non-ambiguous meaning: it changes the Weight and MPG values of the objects to maintain the consistency of the relationship between the data set and its representation. In this example, the data objects with a 0 MPG can be edited to reflect plausible values. Similarly, editing other graphic attributes such as color could be unambiguously translated into data attribute changes. Plot views can be used to edit category attributes, if the various categories are spread on one of the axes.

If the user wishes to edit an individual attribute with increased precision and in a separable manner, a histogram representation provides the appropriate visualization. Figure 2 shows a bar chart representing all the values for a given attribute. Resizing one or more bars can be performed with regular resize interactors without accidentally touching other attribute values. Furthermore, high resolution is obtained using precision resize interactors. Their impact on the value change is inversely proportional to the distance between the pointer and the start of the move operation in the direction orthogonal to the value axis.

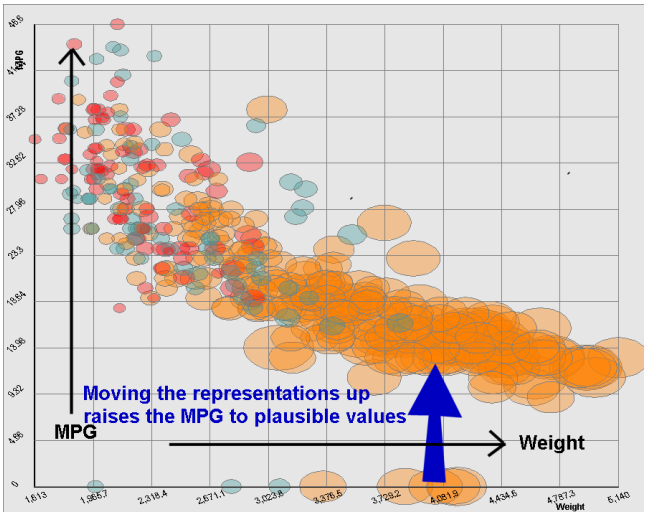


Figure 1: Editing continuous attribute values.

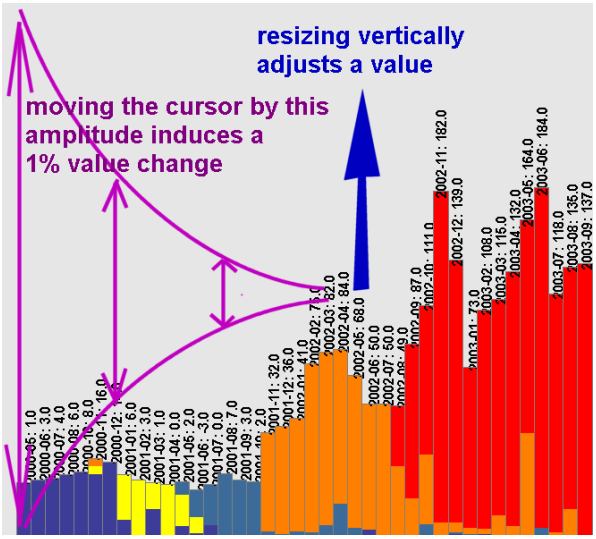


Figure 2: Editing a single value at an arbitrary level of precision.

It should be understood that both editing methods described previously are implemented by the same interactor. In the histogram view, formulas that map horizontal position and size depend on local variables. They are therefore not invertible. Hence, the resize interactor does not let this value change.

Operations in Discovery are fully logged at the semantic level. This enables many undo/redo operations and the setting and comparing of data set snapshot images at various states of the editing process. This lets the user compare alternative scenarios at will to perform "what-if" analyses.

Even though they are fairly straightforward to provide, these features are rarely present in visual analysis tools, perhaps under the scientific premise that good observation methodologies must be careful not to interfere with the studied process. Yet, the ability to test hypotheses through direct manipulation and to obtain immediate visual feedback is certainly one of the most useful tools to enrich the idea creation process and take full advantage of visual data analysis.

## DIRECT MANIPULATION OF BUG REPORTS

The examples we have introduced so far do not move beyond the realm of data analysis. They merely show new features that could improve these processes. After data analysis tasks comes the need to make and execute decisions. The most successful use of our framework so far has been a classical application: defect reports management in a software development firm. This application is now used daily by the development teams. Some of its features can be evaluated in an online demonstration [13]. It is a web application that includes a database holding the bug reports and a set of applets and html forms that address the needs of the various actors who manage bug reports. These actors are:

- Technical support, who browse existing reports and add new reports.
- Team leaders, who assign priorities and dispatch them to engineers.
- Developers and QA engineers, who refine descriptions, fix bugs, and create patches and verification scripts.

This data is also used occasionally by management to assess how various products are advancing and by product marketing who investigate the frequent requests and problems found in the software.

A set of views has been defined. Each view presents the bug reports under a specific perspective. Each of these views prompts (in the sense of a Gibsonian "affordance") the user for particular actions that can be easily done in the given view. A user chooses the view that matches the task to be done, then changes the view definition according to his or her evolving activity.

The dataset to use and the views to be displayed are defined interactively in the views builder and then loaded into a generic applet. Available actions are deducted from the view definition. All interaction with the dataset is done with the regular graphic manipulation tools and contextual menus whose semantic is derived from the view definition.

## IN CONTEXT ATTRIBUTE EDITING

Figure 4 presents the applet showing all the currently open bug reports for a given product. These reports are presented in a grid, grouped by priority from low to critical. A grid is preferable to a treemap in this view as it emphasizes all categories irrespective of the number of items they contain, rather than showing only the big categories. One can identify 5 priorities and their respective bug counts. The color matches the date the report was entered: one old unassigned bug is visible in the lower left corner as a dark rectangle.

The view shown in Figure 4 allows bug priority assignment. It is used primarily by team leaders or project managers, both as a way to assess the current situation and as a means to perform priority assignments.

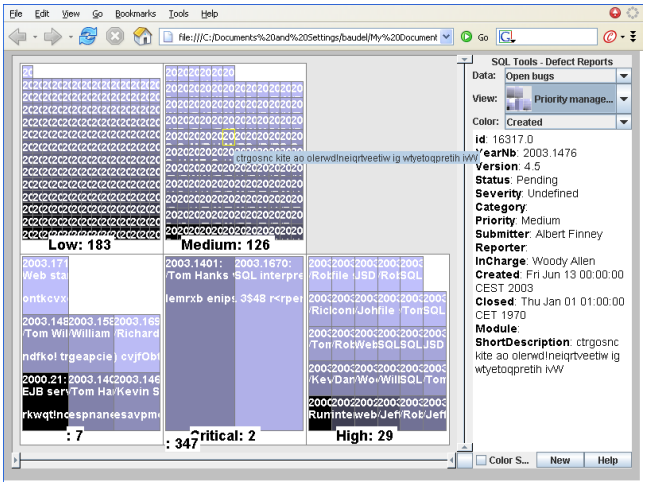


Figure 4: Open bug reports grouped by priority.

Moving the cursor over each bug report displays its summary. The column on the right hand side presents more detailed information. Clicking on a bug report brings up a regular editing form. This is not the primary way to use this view: a contextual pop-up menu attached to each bug report allows the user to move a report from one priority category to another (Users prefer this method over drag and drop). In this case, the user can set the priorities of the 7 unassigned bug reports in this view using simple menu selections. All these priority assignments are performed while viewing the whole context, which helps to minimize errors. This view has enabled users to fix many anomalous priority assignments the very first time it was used.

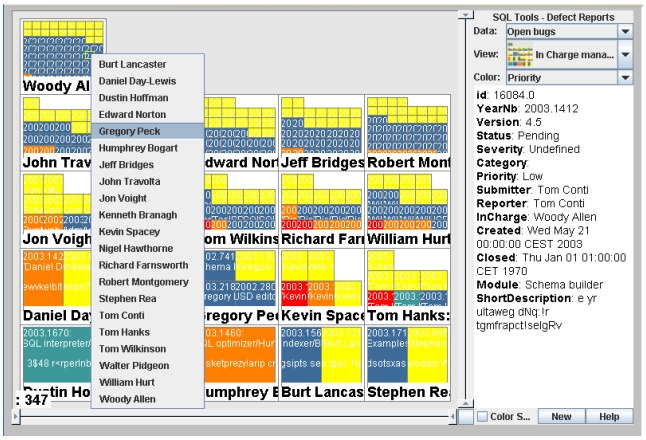


Figure 5: Open bug reports per person in charge.

This mechanism can be used for all the enumerated attributes in the bug reports. Using the same type of view with different partitioning parameters, a project manager can assign bugs to team members or software modules, and keep track of the workload of each developer. In Figure 5,

the data shown in Figure 4 is displayed grouped by person in charge and colored by priority. This view is used daily to assign new bugs and reassign bugs from developers under high pressure.

In the current application, about forty views have been defined to address the large amount of data management tasks. Some other tasks, such as entering an initial report or refining a description still need to be done using regular text forms. Most of the views used so far are grids or treemaps, because our application is mainly used to edit categorical attributes.

The important concept to be retained from this example is that the meaning of the user's actions and the popup menu labels are derived from the view settings, and are not programmed.

### CREATING, DELETING OBJECTS AND ATTRIBUTES

Editing individual values is only part of the required operations on a data set. One may also need to add or remove objects or the attributes of those objects. To illustrate how to perform these operations graphically and in context, we choose another application domain: graphical editing of decision tables. Decision tables are sets of elementary rules that have the same structure, but differ from each other by a few varying elements, such as:

"if driver is in **CA** and **between 21 and 30** and is a **Single Male**, then his car insurance premium is **$300**"

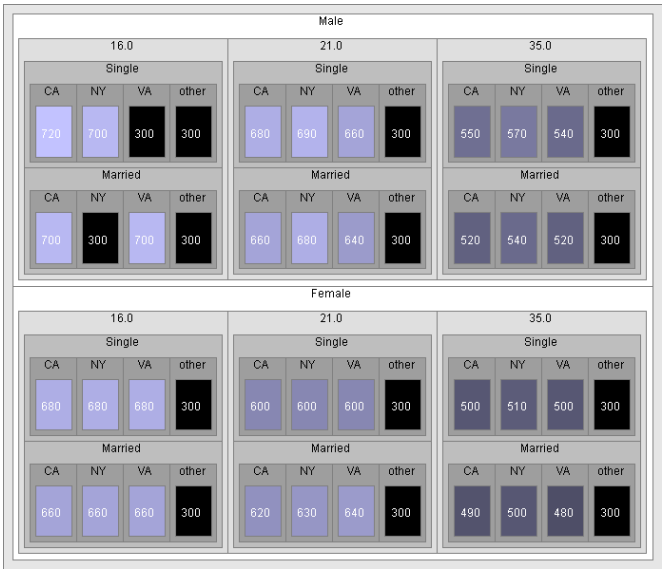


Figure 6: A decision table.

The elements and their domains define the columns in a data table whose rows are built from the Cartesian product of all the possible values for each varying element. There are quite a few limitations in current decision table editing methods that need to be addressed. In our application, decision tables are represented by treemaps whose depth levels materialize each varying element type (Figure 6).

Using a color to represent a varying attribute, in this case the insurance premium, enables rapid identification of the global patterns of the decision table: here, color is related to the premium's value, from low (dark) to high (light). Because treemap hierarchy levels are decoupled from table column ordering, remapping the order of the partitioning allows us to highlight the cost structure, or to ease the selection of parts of the table to focus on (Figure 7).


Figure 7: The decision split by state, age, gender and marital status.

Let us assume the need to add a new state (CT) to the decision table using the values from the VA state as a template. In the first view (Figure 6), as in existing implementations of decision tables, this would require working in each gender and age category to select and duplicate the values of the VA category. Because this operation is tedious to perform graphically, it is usually performed using a specific dialog, and not through direct manipulation. In the reordered view (Figure 7), selecting and duplicating the VA state is done using one selection and one single action.

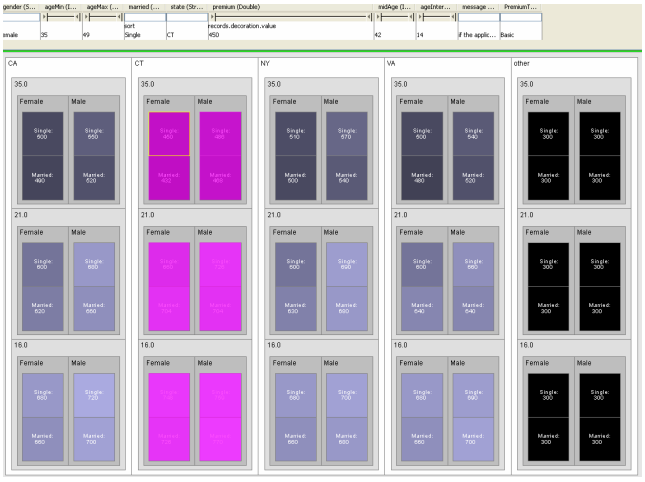

Figure 8: The decision table with one state (CT) added.

Our system also enables the user to edit one or more attributes of the selection in context. First, the duplicated objects are selected and have their state attribute set to "CT" col-

lectively. Entering an expression for the premium attribute, such as "if the age is above 31 then the premium is multiplied by 1.1, else it is multiplied by 0.9" allows one to enter the particularities of this state's premium globally, instead of having to edit the rows one by one (Figure 8).

Other data manipulation actions include delete selected objects, and add or remove attributes to the selection. Using only five elementary actions: change an attribute value, add or remove objects, and add or delete attributes, applied either to a single object or to a group of objects, our system provides the ability to create and manage tabular and hierarchical data structures of arbitrary size and complexity. The interaction is always in context, generic, and requires mastering only a few basic operating principles.

## INTERACTIVE PATTERN SPECIFICATION

A Mapping between an abstract data set and a graphic representation can be used to define a reverse mapping between the spatial layout of the display and attribute domains. This can be used for other purposes. For example, Figure 9 shows statistics about cars in a small multiples view, where the MPG is plotted against the weight of the car, split according to the car's origin.

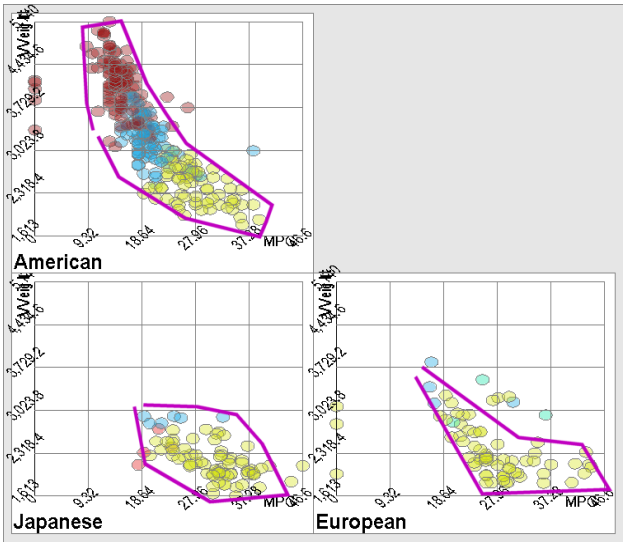


Figure 9: MPG against car weight for a set of cars from three continents.

The 3 polygons, drawn with a polygonal selection tool, define the approximate area that corresponds to the cars that follow the correlation between weight and MPG. The area delimited by those polygons can be interpreted as the left part of a rule in the following form:

```
IF the car's origin is America
   AND its pair (MPG, Weight)
       is inside the polygon
       ((x1,y1),(x2,y2)... (xn,yn))
OR the car's origin is Japan
   AND its pair (MPG, Weight)
       is inside the polygon
     ((x1,y1),(x2,y2)... (xn,yn))...
OR the car's origin is Europe
    AND...
THEN { do something }
```

This rule is fairly difficult to verbalize naturally. However, its graphical representation is much more intuitive. We envision this sort of tool being used for fraud detection or for alarm filtering, for example, to isolate outliers in monitored data streams. This example reveals other possible applications of flexible and reversible mappings between a data set and a visual representation.

## A GENERAL APPROACH TO INTERACTIVE DATA MANIPULATION

The examples we have introduced all contribute a piece of the general workflow that we will now detail. We propose an interaction model for the manipulation of large, abstract data sets based on the first two principles of direct manipulation. This model provides access and modification of the objects with the 3 step workflow already introduced:

1. **Adjust** the view settings to make the representation suit the operations to be performed.
2. **Select** the objects or groups of objects of interest by means of regular or extended selection mechanisms.
3. **Perform** one of the five editing actions: change value, clone, delete, add or remove attributes.

All these steps can be performed in arbitrary order and are detailed below.

### Adjusting the view

This phase allows the user to specify the context of operations and keep this context at hand. This is the innovation in our interaction model that complements regular direct manipulation. While they do not try to provide immediately understandable displays, varying representations show more information in concise ways and highlight chosen portions better. This phase is made possible by the flexible engine that easily creates views of various kinds, and allows these views to adapt to the context locally. Rather than requiring the user to figure out the features of the visualization engine, our system provides 3 types of predefined views: overviews, maps and graphs. These views can be customized further by crossing their main features with one another at varying levels of the partitions.

#### *Overviews*

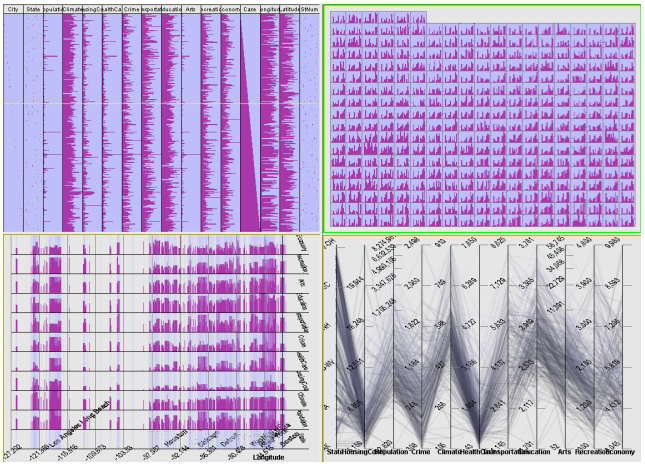

Figure 10: Some predefined overviews.

Overviews give an overall glimpse of a database. They are used to check for gross inconsistencies such as missing data. They are also used to detect direct correlations between pairs of attributes to get a sense of all the attribute domains. Otherwise, they tend not to be very useful for direct editing. Figure 10 shows some of the most useful overviews.

***Maps and grids***

These are space-filling views of various kinds such as grids and treemaps. They enable the user to split the data set and group objects by categories and subcategories. They are also used to focus in on the groups of interest using topological (focus-in) or geometrical (zoom-in) navigation. As we have shown before, they appear particularly useful for editing categorical values in context (Figure 11).

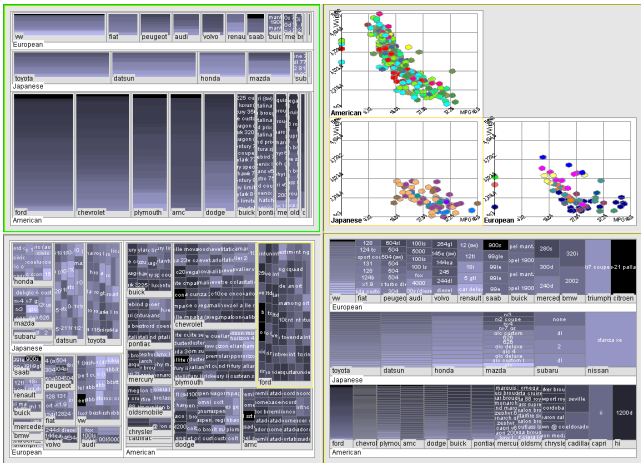

Figure 11: Some of the map views.

***Charts***

Charts, histograms and time-based views are used mainly to display and edit data attributes whose domain is continuous. They can also be used to edit categorical data. We offer a range of the views commonly found in charting packages, as shown in Figure 12.

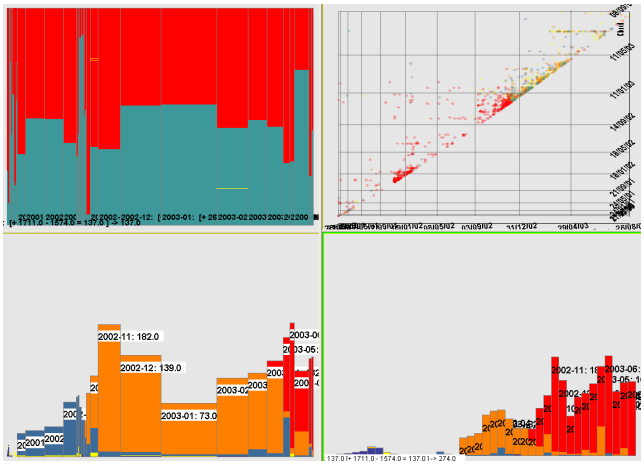


Figure 12: Continuous values representation.

***Navigation tools***

As with all visualization systems, navigation tools of various kinds are needed to refine the presentation. Geometric zooming and panning can be performed with range sliders. Topological navigation can be performed by focusing on groups of objects or by brushing and linking between views. Algebraic navigation operators such as selection, merge, join and projection are to be implemented in future releases.

**Selecting the objects of interest**

One of the particularities that stems from decoupling data objects from their representation is that we have to distinguish two levels of selection. *Graphical selection* deals with selecting object representations and is local to the view. S*emantic selection* enables the software to deal with the objects themselves and therefore propagates to the other views of the same data.

A graphical selection is a tree of graphic objects. Selection is performed using the interactors found in drawing packages: click (with Shift and Control/Click), select area, select groups, select by expression dialog, or transfer from the semantic selection. Graphical selection includes graphical objects which represent groups of objects as well as non-semantic objects (pure decorations). Operations on the graphical selection are not interpreted semantically.

A semantic selection (called *marking* hereafter) is a set of row indices in the data table. Its visual feedback is provided by highlighting (in magenta) the graphical objects that have a one-to-one relationship with a row of the dataset (see Figure 6). Marking is performed by transfer from the graphic selection, set operations (union, intersection, inverse), or by entering a Boolean expression evaluated for all the rows. All editing operations performed on the semantic selection are interpreted as data modifications.

While this decoupling could appear to be an extraneous sophistication, it is needed because precise semantic selection often needs to be performed by combining multiple graphical selections from multiple views and turning them progressively into a unique semantic selection.

For example, in a web server log analysis application, one can select all the page hits by web crawlers to filter them out. This is performed in a view presenting all the requests partitioned by page request. The user selects all the page hits containing a reference to the “robots.txt” file (this is the typical signature of a web crawler) and marks them. That is, the user converts them into a semantic selection as shown in Figure 13: the top left corner in magenta groups all the page hits to the “robots.txt” file.

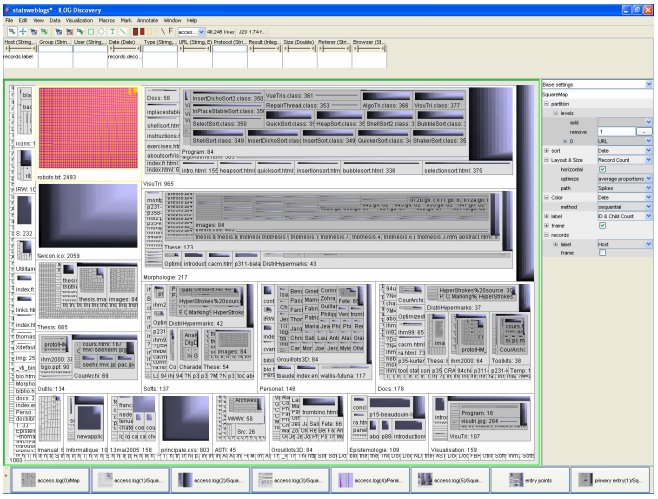

Figure 13: Log file of a small web site, 100000 hits.

In another view, the same data is partitioned by IP addresses. The groups that show a marked object inside them correspond to visits by web crawlers. The marks obtained from the view in figure 13 are converted back into a graphical selection; this graphical selection is converted to a selection of the containing groups (select parents action), which in turn is converted to all objects contained in the selected groups (select children action) as shown in Figure 14. The resulting graphical selection then is converted back into marks and all the page hits from web crawlers can be deleted using the delete command.

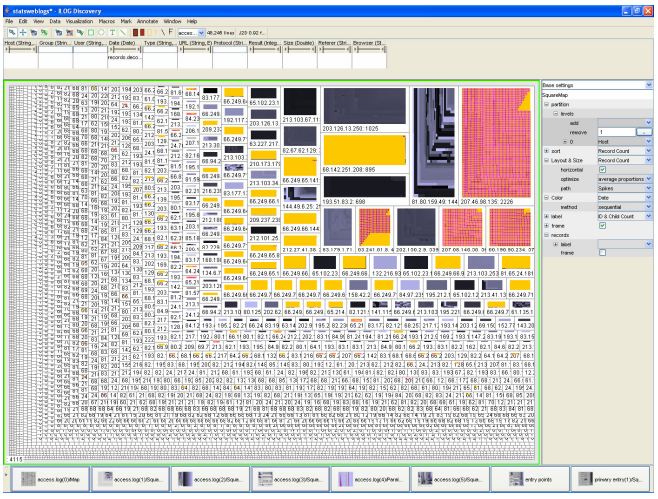

Figure 14: Same Log partitioned by IP addresses. Graphical selection appears in yellow.

Only 8 atomic and generic actions are required for this operation. A similar query in SQL requires a self-join and the use of a negation, and therefore the creation of an intermediate database view, which is definitely out of reach of the average SQL user. This workflow is a typical direct manipulation procedure for creating sophisticated queries using fast, incremental, and reversible actions.

### Editing the data

As mentioned before, any sort of hierarchical data structure can be created and managed in our application, once a start singleton element is provided. It is not difficult to figure this out when we consider the five elementary actions we provide:

1. **Edit** a value for one or more selected objects
2. **Clone** some selected objects
3. **Delete** some selected objects
4. **Add** some attributes to one or more objects
5. **Remove** some attributes from one or more objects

Relational data structures and general graph management would require the additional ability to link and unlink objects. This has not yet been studied. Value changes, the most frequent operations, deserve some attention. The other operations are handled rather trivially.

#### *Value changes*

As seen from our examples, we distinguish between categorical value changes, performed in space-filling views, and continuous value changes, performed in charts and 2D graphs views. Other types of values such as text or binary data still have to be handled with specialized editors. To address all editing needs, a side view presents all the available attributes and enables the user to enter a new value or an expression that will be evaluated for all marked objects. This departs somewhat from direct manipulation but is most convenient.

#### *Implementation using linear state dataflows*

The most innovative feature in terms of data editing is the ability to invert the mapping from the data to the representation and thus deduce semantic changes from graphical operations. This is made possible by the linear-state dataflow model, which identifies precisely the relationship between each data value and the graphic attributes of the shapes and groups displayed in a view.

Assuming knowledge of [1], here is how inverse mapping is performed. The expressions that define graphical attribute values that we might want to invert contain:

- **Pure constants.** These are not linked to the data, and are not invertible expressions. For example, the move interactor does not do any reverse mapping when the user attempts to move an object (such as a histogram bar) whose vertical position is set to the expression "0".
- **Data attribute values**. For example, the height is mapped to a "Revenue" column. If there is only one such data attribute in the expression, an inversion of the mapping formula provides a direct inverse mapping. This happens in 2d plots or when changing the height of histogram bars.
- **Local variables**. These are values defined within the scope of an enclosing dataflow node. Our model uses only two sorts of nodes and therefore only two cases to consider:

  a. **Grouping/partitioning nodes**: A local variable holds a value computed from all the values

held in a group, such as the sum of a "population" attribute. When possible, inversion uses this value as a weight, making changes to individual data values relative to the value computed for the whole group.

b. **Ordering/sort nodes**. There are two subcases: the sort criterion is endogenous: objects are sorted according to an attribute's value, or it is exogenous: objects are taken in the order they come, or some other ordering scheme. For endogenous nodes, the change of a graphic attribute is interpreted as a change in the order in which the objects come, and thus a change of the data attribute that specifies the ordering relationship. This works well for enumerated or discrete valuations. For continuous attributes, we assign the middle between the preceding and following data value, which may not satisfy all uses. For exogenous nodes, the attribute value change is considered ambiguous. However, more refined analysis may be conceivable.

When only one of the above rules can be used, inversion is unambiguous. When two or more rules apply, we might want to ask the user to arbitrate. From a usability standpoint, we do not believe it is useful to pursue too sophisticated an analysis of the mapping formulas, as we consider the predictability of a user's actions to be a major asset of direct manipulation. Hence, graphical actions are performed only when their interpretation does not raise a possible conflict, which is the case in most simple views.

#### *Other actions*

The clone objects, delete objects, add attribute, and remove attribute actions are performed using menu or keyboard shortcut actions. The last two require an extra dialog to specify the attribute to be removed or the characteristics of the new attribute. Our current data model is a table, which means that current attributes are added to the whole table and not just the marked objects. This does not affect the capabilities of the interaction model as a whole.

We did consider using the generic cut/copy/paste functions to perform editing operations. However, these have been shown to be somewhat confusing. They can be interpreted at three levels: the graphical objects level (copy image or copy graphic objects), the mapping level (copy data to view mappings), or the semantic level (copy the data objects). This problem is found in many office automation applications that have resorted to separating these features rather than using modes to disambiguate the various possible interpretations of these generic functions.

## EVALUATION AND USER FEEDBACK

While numerous limitations exist in the current prototype, the interactions described here are already in daily use. More in-house applications are being developed based on the same model, such as a sales-pipeline monitoring and dispatching tool and a technical support resources manager. The fact that our hand-made prototype has usefully complemented a commercially-developed defect reports management system is, for us, proof of the usefulness of our system and proposed interaction paradigm. Controlled user experiments would definitely help us assess how to further improve on our design. However, the features we provide enable a very new workflow, which explains both why it has been adopted as a complement to the existing tool, and why a controlled comparison is difficult. The following feedback may explain this better:

- "When assigning new reports to developers, I am not always sure who is in charge of a particular module. Rather than look for this information, I assign the color of the viewed reports to the module name and find the person in charge immediately". The ability to change the view parameters in many unplanned ways creates new facilities to support tasks that involve judgment and knowledge of many facts contained in the data set.

- "I realized that two developers had a back-log of over 100 bugs. I figured there was little point in continuing to assign newly opened bugs, and moved on to re-prioritize and dispatch the backlog". By having the full context at hand, the proper action follows immediately from the observation, rather than coming from a separate analysis activity.

We plan on further refining the implementation based on the design principles detailed in this article. For example, a major issue appears to be concurrent editing: as two users can simultaneously access and edit many objects in one single action, the chances of conflicts are greatly amplified by our interface. To help with this issue, we envision providing real-time visual feedback of the other users' actions through techniques similar to those of GroupDesign [3].

## CONCLUSION

In this article, we have extended the principles of direct manipulation to enable the editing of large abstract datasets for which no intrinsic or conventional graphic representation exist. This extension relies on the flexibility of a generic visualization system that allows the presentation of data in the most appropriate perspective for the tasks to be performed and the ability to reverse its mappings.

The proposed model has been derived from observation and from examples of use that we have generalized into an interaction model defined as a set of design guidelines. These guidelines can be summarized as:

1. Provide a flexible visualization system that allows rich and precise **view adjustments**, as well as geometric, topological, and algebraic navigation.
2. Provide semantic and graphical **selection methods**.
3. Provide the **five editing actions** (change value, clone/delete objects, add/remove attributes) as fast, reversible, and incremental operations.

The interaction techniques used are derived directly from widespread graphical interaction techniques such as select, move and delete, which lets us assume a decent level of usability. The most novel part of the interaction model, view adjustments, is simplified in our system by providing many predefined view types chosen according to the most

frequent uses found for our system. The major contribution of our prototype lies in the inference of the semantics of the user's actions from the view settings. Coupled with easy adjustment of the views, this provides predictable and concise interaction methods in line with the principles of direct manipulation.

Our prototype has been implemented in the ILOG Discovery Information Visualization framework, freely available for evaluation purposes [13] and some of the features are already in daily use in some internal applications.

The Information Visualization community sometimes expresses frustration over the slow market adoption of techniques and tools provided by its research community. The success of our internal prototypes leads us to conjecture that providing generic methods to *edit* the data in context, using bulk information manipulation techniques, brings novel applications and uses for this research domain, beyond the limiting focus of visual data analysis. This allows us to foresee these concepts being put to use to enable the direct manipulation of masses of information in the spreadsheets of the XXI$^{st}$ century.

**Acknowledgments**

Thanks to I. Cox and W. McNeill for proofreading. Julian Payne, Patrick Megard and many in ILOG R&D provided invaluable insights during the development of this work.

**REFERENCES**

1. Baudel, T. *Canonical Representation of Data-Linear Visualization Algorithms and its Applications*. ILOG research report, 2002-03, available at http://techreports.ilog.com. Published in French as *Visualisations compactes: une approche déclarative pour la visualisation d'information*, Proceedings of the 14th French-speaking conference on Human-computer interaction (IHM02), p.161-168, ACM International Conference Proceeding Series; Vol. 32.
2. Baudel, T. *Browsing through an information visualization design space*. Extended abstracts of the 2004 conference on Human factors and computing pp. 765 - 766, ACM Press, 2004.
3. Beaudouin-Lafon, M., Karsenty A., *Transparency and Awareness in Real-Time Groupware Systems*, Proceedings of the ACM Symposium on User Interface Software and Technology (UIST'92), 1992, pp.171-180.
4. Casner, S.M. *A Task-Analytic Approach to the Automated Design of Graphic Presentations*. ACM Transactions on Graphics, 10(2), 111-151. 1991.
5. Chi, Ed., *A Framework for Visualizing Information*. 2002. Kluwer Academic Publishers, Netherlands.
6. Cypher, A. et al. *Watch What I Do: Programming by Demonstration*. The MIT Press. 1993
7. Derthick, M., Kolojejchick, J. A., and Roth, S.. *An Interactive Visual Query Environment for Exploring Data*. Proceedings of the ACM Symposium on User Interface Software and Technology (UIST '97), ACM Press, October 1997, pp 189-198.
8. Derthick, M. and Roth, S. *Enhancing Data Exploration with a Branching History of User Operations*. Knowledge Based Systems, 14(1-2):65-74, March 2001.
9. Fekete, J.-D. *The InfoVis Toolkit*, Proceedings of the InfoVis '04 conference, pp. 167-174, 2004.
10. Heer, J., Stuart K. Card, and James A. Landay. *Prefuse: a toolkit for interactive information visualization*. In proceedings of the CHI 2005, Human Factors in Computing Systems, conference, 2005.
11. Hibbard, B. *VisAd*, http://www.ssec.wisc.edu/~billh/visad.html.
12. Karsenty , S., James A. Landay , Chris Weikart, *Inferring graphical constraints with Rockit*, Proceedings of the conference on People and computers VII, p.137-153, January 1993, York, United Kingdom
13. ILOG, *ILOG Discovery for direct manipulation database editing*. Interactive demonstration available at http://www2.ilog.com/preview/Discovery/ Dec. 2003.
14. Kolojejchick, J. A., Roth, S. F., and Lucas, P. *Information Appliances and Tools in Visage IEEE Computer Graphics and Applications*, Volume 17, Number 4, July/August 1997, 32-41.
15. Lieberman, H. (Ed.) *Your Wish is My Command: Programming by Example*. San Francisco: Morgan Kaufmann. 2001.
16. Liskov, B. et al. *CLU reference manual*. In Goos and Hartmanis, editors, Lecture Notes in Computer Science, volume 114. Springer-Verlag, Berlin, 1981.
17. Roth, S. F., Chuah, M. C., Kerpedjiev, S., Kolojejchick, J. A., and Lucas, P. *Towards an Information Visualization Workspace: Combining Multiple Means of Expression.* Human-Computer Interaction Journal, Volume 12, Numbers 1 & 2, 1997, 131-185.
18. Schneiderman, B., *Direct Manipulation: a Step beyond programming languages*. IEEE Computer, Vol. 16, No. 8 (August 1983), pp.57-69.
19. Schroeder, W., K. Martin and Lorensen, B. *The Visualization Toolkit*, An Object-Oriented Approach To 3D Graphics, Prentice-Hall.
20. Stolte, C., D. Tang, P. Hanrahan, *Polaris: A System for Query, Analysis, and Visualization of Multidimensional Relational Databases*, IEEE transactions on visualization and computer graphics vol. 8, no. 1; January-March 2002, pp. 52-65
21. Thompson, D., Braun, J. & Ford, R. *OpenDX: Paths to Visualization*, Visualization and Imagery Solutions, Inc., 2001.
22. Wilkinson, L. *The Grammar of Graphics*. New York: Springer Verlag, 1999. pp. 66-78.